

IoT Project - wearable Health Care Module Module

Problem Statement : The IoT Device consisting of temperature sensor and heartbeat sensor using the Arduino and integrating the woman safety feature using and location services on SOS button.

Here's a breakdown of the requirements and a possible approach:

1. Hardware Components:

- Temperature sensor (e.g., DS18B20, DHT11, DHT22)
- Heartbeat sensor (e.g., pulse sensor)
- GPS module (for location services)
- Push button (for SOS functionality)
- Wi-Fi or cellular module (for internet connectivity)

2. Software Components:

- Arduino IDE (Integrated Development Environment)
- Libraries for interfacing with sensors (e.g., OneWire, DHTlib, PulseSensorPlayground)
- Libraries for GPS and location services (e.g., TinyGPSPlus)
- Libraries for internet connectivity (e.g., WiFi.h, GSM.h)

3. Functionality:

- Temperature Monitoring:
 - Read temperature data from the temperature sensor periodically.
 - Process and store the temperature data for further analysis or transmission.
- Heartbeat Monitoring:
 - Read heartbeat data from the pulse sensor periodically.
 - Process and store the heartbeat data for further analysis or transmission.
- Woman Safety Feature:
 - Implement an SOS button using the push button.
 - When the SOS button is pressed, retrieve the current location using the GPS module.
 - Send an alert message with the current location to a predetermined emergency contact or service using internet connectivity.
- Data Transmission:
 - Establish internet connectivity using the Wi-Fi or cellular module.
 - Transmit the collected temperature, heartbeat, and location data to a remote server or cloud platform for further processing, analysis, or storage.

- User Interface:
 - Optionally, incorporate an LCD display or other output devices to provide real-time feedback or information to the user.

4. Integration and Testing:

- Integrate all the hardware components with the Arduino board.
- Write the Arduino code to handle sensor data acquisition, SOS functionality, location services, and data transmission.
- Test individual components and functionalities separately before integrating them into the complete system.
- Perform thorough testing and debugging of the complete IoT device.

5. Deployment and Maintenance:

- Deploy the IoT device in the desired environment or location.
- Implement a mechanism for over-the-air (OTA) firmware updates or remote configuration changes, if required.
- Establish a monitoring and maintenance plan for the deployed devices.

This problem statement involves various aspects, including hardware integration, software development, sensor data acquisition, location services, internet connectivity, and data transmission. It requires a solid understanding of Arduino programming, sensor interfacing, GPS/location services, and networking concepts.

Note: Depending on the specific requirements and constraints of the project, additional features or modifications may be necessary. Additionally, proper security measures should be implemented to ensure data privacy and protection, especially when transmitting sensitive information over the internet. The problem statement you provided involves designing an IoT (Internet of Things) device that integrates multiple sensors and features for monitoring health and safety. Here's a breakdown of the requirements and a possible approach:

1. Hardware Components:

- Arduino board (e.g., Arduino Uno, Arduino Nano)
- Temperature sensor (e.g., DS18B20, DHT11, DHT22)
- Heartbeat sensor (e.g., pulse sensor)
- GPS module (for location services)
- Push button (for SOS functionality)
- Wi-Fi or cellular module (for internet connectivity)

2. Software Components:

- Arduino IDE (Integrated Development Environment)
- Libraries for interfacing with sensors (e.g., OneWire, DHTlib,

PulseSensorPlayground)

- Libraries for GPS and location services (e.g., TinyGPSPlus)
- Libraries for internet connectivity (e.g., WiFi.h, GSM.h)

3. Functionality:

- Temperature Monitoring:
 - Read temperature data from the temperature sensor periodically.
 - Process and store the temperature data for further analysis or transmission.
- Heartbeat Monitoring:
 - Read heartbeat data from the pulse sensor periodically.
 - Process and store the heartbeat data for further analysis or transmission.
- Woman Safety Feature:
 - Implement an SOS button using the push button.
 - When the SOS button is pressed, retrieve the current location using the GPS module.
 - Send an alert message with the current location to a predetermined emergency contact or service using internet connectivity.
- Data Transmission:
 - Establish internet connectivity using the Wi-Fi or cellular module.
 - Transmit the collected temperature, heartbeat, and location data to a remote server or cloud platform for further processing, analysis, or storage.
- User Interface:
 - Optionally, incorporate an LCD display or other output devices to provide real-time feedback or information to the user.

4. Integration and Testing:

- Integrate all the hardware components with the Arduino board.
- Write the Arduino code to handle sensor data acquisition, SOS functionality, location services, and data transmission.
- Test individual components and functionalities separately before integrating them into the complete system.
- Perform thorough testing and debugging of the complete IoT device.

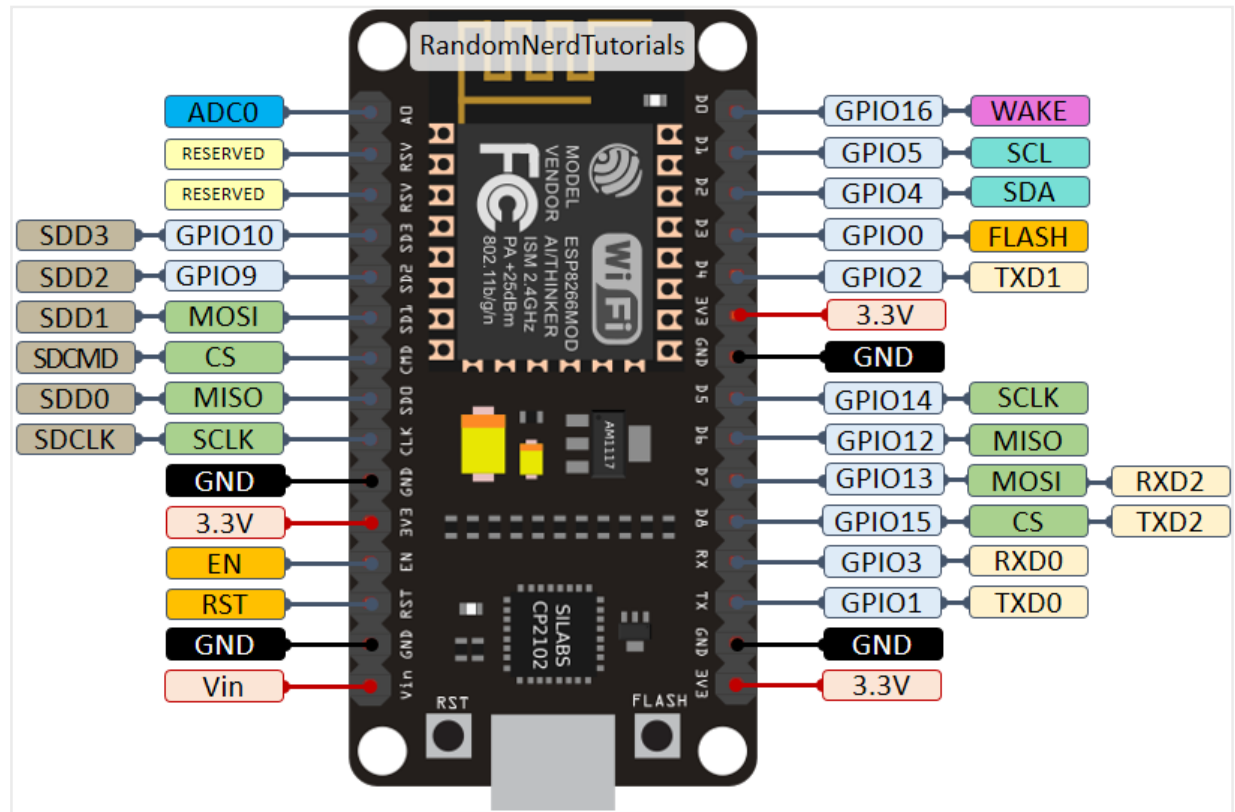
5. Deployment and Maintenance:

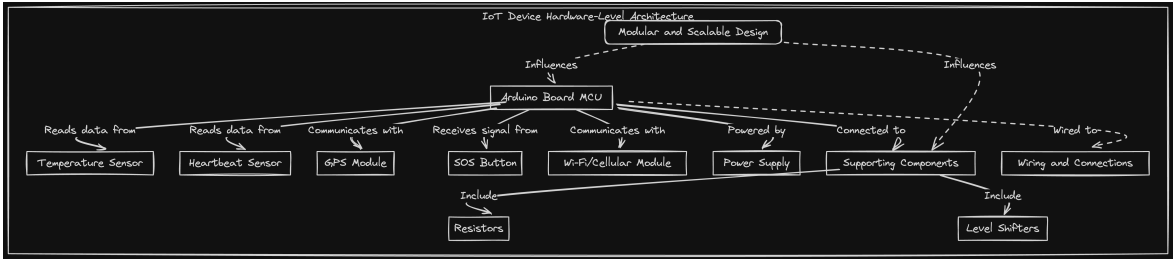
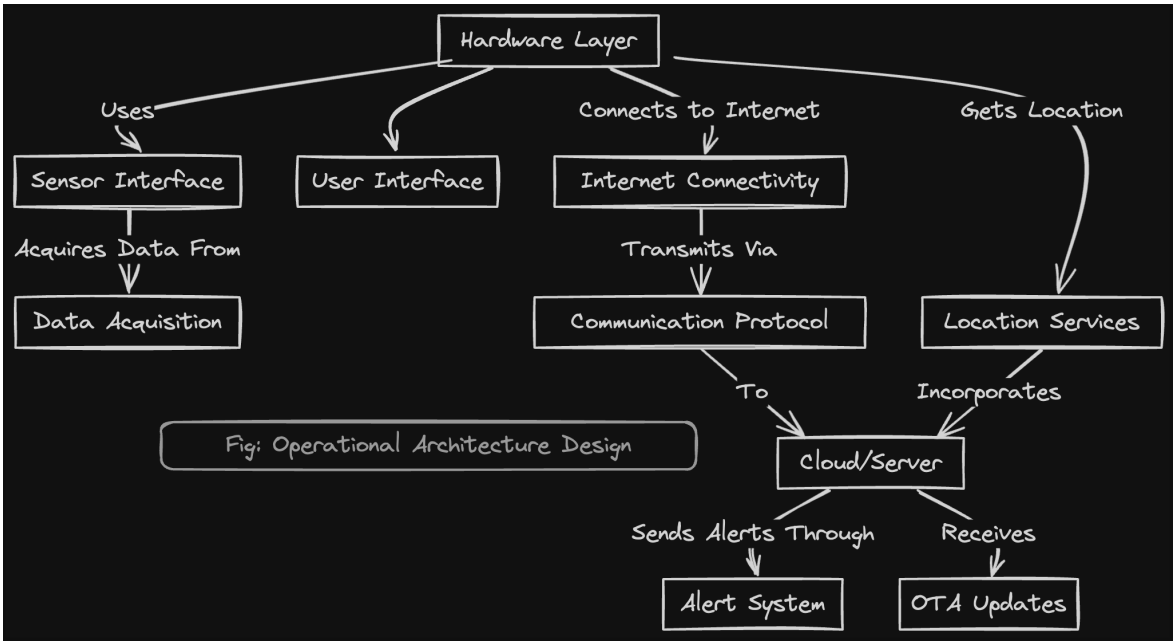
- Deploy the IoT device in the desired environment or location.
- Implement a mechanism for over-the-air (OTA) firmware updates or remote configuration changes, if required.
- Establish a monitoring and maintenance plan for the deployed devices.

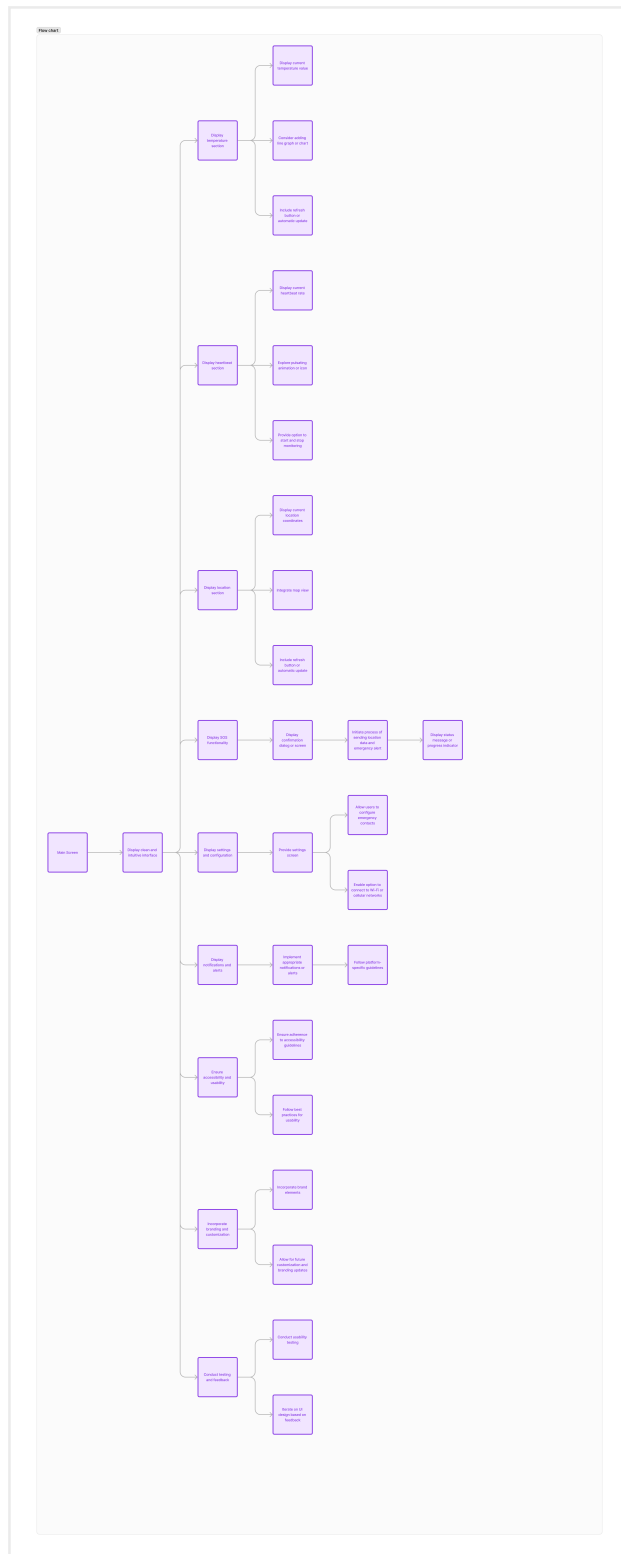
This problem statement involves various aspects, including hardware integration,

software development, sensor data acquisition, location services, internet connectivity, and data transmission. It requires a solid understanding of Arduino programming, sensor interfacing, GPS/location services, and networking concepts.

Note: Depending on the specific requirements and constraints of the project, additional features or modifications may be necessary. Additionally, proper security measures should be implemented to ensure data privacy and protection, especially when transmitting sensitive information over the internet.







```

#include <ESP8266WiFi.h>
#include <WiFiClient.h>
#include <OneWire.h>
#include <DallasTemperature.h>
#include <ThingSpeak.h>

```

```
// WiFi credentials
const char* ssid = "YOUR_WIFI_SSID";
const char* password = "YOUR_WIFI_PASSWORD";

// ThingSpeak settings
const char* server = "api.thingspeak.com";
const char* apiKey = "YOUR_THINGSPEAK_API_KEY";
const unsigned long postingInterval = 10 * 1000; // Posting
interval in milliseconds (e.g., every 10 seconds)

// DS18B20 temperature sensor setup
#define ONE_WIRE_BUS D4 // Pin D4 for one-wire communication
OneWire oneWire(ONE_WIRE_BUS);
DallasTemperature sensors(&oneWire);

// Heart rate sensor setup
const int heartRatePin = A0; // Analog pin for heart rate
sensor

WiFiClient client;

void setup() {
  Serial.begin(115200);

  // Connect to WiFi
  Serial.println();
  Serial.println("Connecting to WiFi...");
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("");
  Serial.println("WiFi connected");

  ThingSpeak.begin(client); // Initialize ThingSpeak

  // Start DS18B20 temperature sensor
  sensors.begin();
}

void loop() {
  if (WiFi.status() == WL_CONNECTED) {
```

```
float temperature = readTemperature();
int heartRate = readHeartRate();

// Update ThingSpeak fields
ThingSpeak.writeField(YOUR_THINGSPEAK_CHANNEL_ID, 1,
temperature, apiKey);
ThingSpeak.writeField(YOUR_THINGSPEAK_CHANNEL_ID, 2,
heartRate, apiKey);

Serial.print("Temperature: ");
Serial.print(temperature);
Serial.print("°C, Heart Rate: ");
Serial.println(heartRate);

delay(postingInterval);
} else {
    Serial.println("WiFi not connected");
}
}

float readTemperature() {
    sensors.requestTemperatures(); // Request temperature from
DS18B20 sensor
    return sensors.getTempCByIndex(0); // Return temperature in
Celsius
}

int readHeartRate() {
    return analogRead(heartRatePin); // Read heart rate from
analog pin
}

#include <SoftwareSerial.h>
#include <LiquidCrystal_I2C.h>
#include <millisDelay.h>
#include <Wire.h>
#include <SPI.h>
#define RX 12
#define TX 11
String AP = "xxxxxxx"; // CHANGE ME
String PASS = "xxxxxxx"; // CHANGE ME
String API = "xxxxxxxxx"; // CHANGE ME
```

```
String HOST = "api.thingspeak.com";
String PORT = "80";
String field = "field1";
int countTrueCommand;
int countTimeCommand;
boolean found = false;
//boolean pulseFound = false;
int valSensor = 1;
//int finalbpm = 0;
SoftwareSerial esp8266(RX,TX);

    millisDelay timerDelay;
int pulsesensor = 0;
int counter = 0;
int bpm = 0;
int test = 0;
int rate = 0;
int r1,r2,r3,r4,r5 = 0;
LiquidCrystal_I2C lcd(0x27, 16, 2);

void sendCommand(String command, int maxTime, char
readReplay[]) {
    Serial.print(countTrueCommand);
    Serial.print(". at command => ");
    Serial.print(command);
    Serial.print(" ");
    while(countTimeCommand < (maxTime*1))
    {
        esp8266.println(command);//at+cipsend
        if(esp8266.find(readReplay))//ok
        {
            Serial.print("true");
            found = true;
            break;
        }

        countTimeCommand++;
    }

    if(found == true)
    {
        Serial.println("OYI");
        countTrueCommand++;
    }
}
```

```
        countTimeCommand = 0;
    }

    if(found == false)
    {
        Serial.println("Fail");
        countTrueCommand = 0;
        countTimeCommand = 0;
    }

    found = false;
}

void pulse(){
    if (pulsesensor < 70){
        r1 = 1;
        r2 = 1;
        r3 = 1;
        r4 = 1;
        r5 = 1;
        lcd.setCursor(0, 0);
        lcd.print("Heart Rate:");
        lcd.setCursor(2, 1);
        lcd.print("Please Wait");
        delay(6000);
        test = 1;
        timerDelay.start(15000);
        lcd.clear();
        lcd.setCursor(0, 0);
        lcd.print("Heart Rate:");
        lcd.setCursor(2, 1);
        lcd.print("Scanning");
    }

    if (test == 1) {
        if (pulsesensor == 540 && r1 == 1)
        {
            r2 = 0;
            r3 = 0;
            r4 = 0;
            r5 = 0;
            counter++;
            delay(500);
        }
    }
}
```

```
        Serial.print("counter:");
        Serial.println(counter);
    }
    else if (pulsesensor == 530 && r2 == 1)
    {
        counter++;
        delay(500);
        Serial.print("counter:");
        Serial.println(counter);
        r1 = 0;
        r3 = 0;
        r4 = 0;
        r5 = 0;
    }
    else if (pulsesensor == 520 && r3 == 1)
    {
        counter++;
        delay(500);
        Serial.print("counter:");
        Serial.println(counter);
        r2 = 0;
        r1 = 0;
        r4 = 0;
        r5 = 0;
    }
    else if (pulsesensor == 512 && r4 == 1)
    {
        counter++;
        delay(500);
        Serial.print("counter:");
        Serial.println(counter);
        r2 = 0;
        r3 = 0;
        r1 = 0;
        r5 = 0;
    }
    else if (pulsesensor == 500 && r5 == 1)
    {
        counter++;
        delay(500);
        Serial.print("counter:");
        Serial.println(counter);
        r2 = 0;
```

```
        r3 = 0;
        r4 = 0;
        r1 = 0;
    }

    if (timerDelay.justFinished())
    {
        lcd.clear();
        lcd.setCursor(0, 0);
        lcd.print("Heart Rate:");
        lcd.setCursor(2, 1);
        lcd.print("Remove");
        bpm = counter * 8;
        rate = 1;
        test = 0;
        counter = 0;
        Serial.println("bpm:");
        Serial.println(bpm);
        //finalbpm = bpm;
        delay(5000);
        lcd.clear();
    }
}

if (rate == 1)
{
    if (bpm > 60 && bpm < 100 || bpm == 60 || bpm == 100)
    {
        normal();
        rate = 0;
        delay(5000);
        //pulseFound = true;
        bpm = 0;
        lcd.clear();
        lcd.setCursor(0, 0);
        lcd.print("Heart Rate:");
    }
    else if (bpm < 60)
    {
        slow();
        rate = 0;
        delay(5000);
        //pulseFound = true;
        bpm = 0;
    }
}
```

```
        lcd.setCursor(0, 0);
        lcd.clear();
        lcd.setCursor(0, 0);
        lcd.print("Heart Rate:");
    }
    else if (bpm > 100)
    {
        fast();
        rate = 0;
        delay(5000);
        //pulseFound = true;
        bpm = 0;
        lcd.setCursor(0, 0);
        lcd.clear();
        lcd.setCursor(0, 0);
        lcd.print("Heart Rate:");
    }

}

}

void slow()
{
    lcd.setCursor(0, 0);
    lcd.print("Heart Rate:");
    lcd.setCursor(2, 1);
    lcd.print("Slow");
    lcd.setCursor(9, 1);
    lcd.print("BPM:");
    lcd.setCursor(13, 1);
    lcd.print(bpm);
}

void normal()
{
    lcd.setCursor(0, 0);
    lcd.print("Heart Rate:");
    lcd.setCursor(2, 1);
    lcd.print("normal");
    lcd.setCursor(9, 1);
    lcd.print("BPM:");
    lcd.setCursor(13, 1);
    lcd.print(bpm);
}

void fast()
```

```

{
  lcd.setCursor(0, 0);
  lcd.print("Heart Rate:");
  lcd.setCursor(2, 1);
  lcd.print("Fast");
  lcd.setCursor(9, 1);
  lcd.print("BPM:");
  lcd.setCursor(13, 1);
  lcd.print(bpm);
}

void wifi(){
  valSensor = getSensorData();
  String getData = "GET /update?api_key="+ API +"&"+ field
+"="+String(valSensor);
  sendCommand("AT+CIPMUX=1",5,"OK");
  sendCommand("AT+CIPSTART=0,\"TCP\", \"\"+ HOST +"\", "+
PORT,15,"OK");
  sendCommand("AT+CIPSEND=0," +String(getData.length()
+4),4,">");
  esp8266.println(getData);delay(1500);countTrueCommand++;
  sendCommand("AT+CIPCLOSE=0",5,"OK");
}

int getSensorData(){
  return bpm;
}

void setup() {
  Serial.begin(9600);
  lcd.begin();
  lcd.setCursor(0, 0);
  lcd.print("Heart Beat");
  lcd.setCursor(2, 1);
  lcd.print("Sensor");
  lcd.backlight();
  delay(40);
  esp8266.begin(115200);
  sendCommand("AT",5,"OK");
  sendCommand("AT+CWMODE=1",5,"OK");
  sendCommand("AT+CWJAP=\"\"+ AP +"\",\"\"+ PASS +"\",20,"OK");
}

void loop() {

```

```
pulsesensor = analogRead(A0);  
pulse();  
Serial.println(pulsesensor);  
delay(1000);  
wifi();  
delay(100);  
}
```